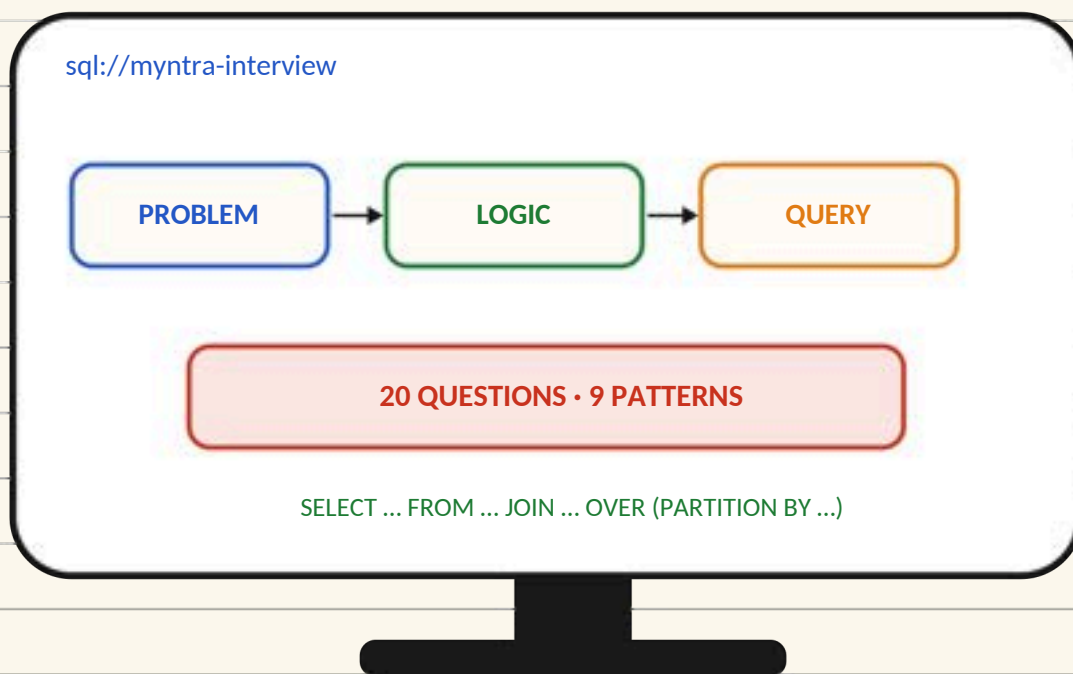


SQL Interview Guide

≈ (Myntra Data Analyst · 20 questions · From problem → logic → query → output) ≈



PROBLEM → LOGIC → QUERY → OUTPUT

YOE: 2-5

CTC: 25+

Window functions

Joins

Optimisation



RQ_Data_Talks



RQanesho203



About This Guide

Twenty real SQL questions from a Myntra Data Analyst interview, rewritten as a clean visual notebook. Read it top to bottom: every question follows the same four beats, so patterns start to repeat and stick.

01 • What is inside

Part	Pages	What you get
Foundations	4 - 5	Execution order of a query and a window-function primer
The 20 questions	6 - 28	Problem, input table, query, expected output, key idea
Pattern map and cheat-sheets	29 - 31	Which technique solves what; MySQL / PostgreSQL / SQL Server; pitfalls
Interview playbook	32 - 34	How to answer, likely follow-ups, summary, corrections log

02 • How every question is laid out

- 1 Problem** — the question exactly as an interviewer would say it, plus the goal in one line.
- 2 Input** — a small sample table so you can trace the logic by hand.
- 3 Query** — a working solution, with alternatives where they matter (ties, dialects, performance).
- 4 Output** — the expected result, followed by the one idea worth remembering.

◆ Simple Analogy

Think of each question as a recipe: the **input table** is the ingredients, the **query** is the method, and the **output** is the finished dish. Learn the method once and you can cook any dish of the same family.

03 • Difficulty legend (our estimate)

Level	Meaning	Typical questions
★	One idea: a join, an aggregate or a subquery	Q1, Q3, Q9, Q11, Q12, Q17
★★	A window function or a careful edge case (ties, NULLs)	Q2, Q4, Q6, Q8, Q10, Q13, Q15, Q16, Q19, Q20
★★★	Needs a trick: islands, month arithmetic or row maths	Q5, Q14, Q18

★ Source note

Questions come from the "Myntra Data Analyst Interview - 20 SQL Questions" PDF (YOE 2-5, CTC 25+). Where a sample query or output needed fixing, it is flagged with a warning box and listed on page 34.

Contents – All 20 Questions

#	Level	Question	Core technique	Page
Q1	★	Second highest salary(no LIMIT / OFFSET/TOP)	Subquery+MAX	6
Q2	★★	Running total of revenue per day	SUM() OVER	7
Q3	★	Employees earning more than their manager	Self join	8
Q4	★★	Top N customers by purchase, ties handled	DENSE_RANK	9
Q5	★★★★	Consecutive login streaks (3+ days)	Gaps and islands	10
Q6	★★	Detect and delete duplicate rows	MIN id / ROW_NUMBER	12
Q7	★★	Optimise a slow multi-join query	Indexes, EXPLAIN, MVs	13
Q8	★★	First order per customer, ties handled	RANK	15
Q9	★	Products that were never purchased	Anti-join	16
Q10	★★	Customers active in every month of a year Same	HAVING COUNT DISTINCT	17
Q11	★	salary as a colleague in the department	Self join / COUNT OVER	18
Q12	★	Department with the highest total salary	GROUP BY + rank	19
Q13	★★	Top 3 orders per customer	Top-N per group	20
Q14	★★★★	Median salary without built-ins	Window + row maths	21
Q15	★★	Pivot sales into month columns	CASE + SUM	22
Q16	★★	7-day moving average per product	Window frames	23
Q17	★	First and last event per user	MIN / MAX	24
Q18	★★★★	Two consecutive months, but not a third	Month index join	25
Q19	★★	Most frequent category per user (12 months)	COUNT + RANK	27
Q20	★★	Rank products by sales, reset every year	RANK PARTITION BY year	28

★ Where to start

Newto window functions? Read pages 4 and 5 first, then do Q2, Q4 and Q8 in that order. Short on time? Q1, Q3, Q8, Q13 and Q5 cover most of what interviewers ask.

Foundations — How a Query Runs

You write **SELECT** first, but the database runs it almost last. Knowing the real order explains why some queries fail and why we wrap window functions in a CTE.

01 • Written order vs execution order

You write: **SELECT** → **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **ORDER BY** → **LIMIT**

The engine runs:



◆ Simple Analogy

A factory assembly line: raw material arrives (**FROM**), faulty parts are removed (**WHERE**), parts are bundled (**GROUP BY**), bad bundles are removed (**HAVING**), each item is ranked (**WINDOW**), then the final box is labelled (**SELECT**) and sorted.

02 • Why the order matters

Clause	Its job	Consequence you will meet
WHERE	Filters rows before grouping	Cannot see aggregates or window results; use HAVING or a CTE
GROUP BY	Collapses rows into groups	Every selected column must be grouped or aggregated
HAVING	Filters groups after aggregation	Q10: HAVING COUNT(DISTINCT month) = 12
WINDOW	Adds a value per row, keeps every row	Runs after GROUP BY , so Q2 can use SUM(SUM(x)) OVER
SELECT alias	Names a column	Not visible in WHERE ; visible in ORDER BY

⚠ Warning

You cannot write **WHERE *rnk* = 1** in the same query that defines *rnk* with **RANK()**. Put the window function in a CTE or subquery, then filter outside. Q4, Q8, Q13, Q19 and Q20 all do this.

Foundations – Window Functions

A window function calculates something across a set of related rows **without collapsing them**. GROUPBY gives one row per group; OVER() gives you the answer on every row.

01 • The syntax in one line

```
function(...) OVER (  
  PARTITION BY col      -- restart the calculation for each group  
  ORDER BY col          -- order rows inside the group  
  ROWS BETWEEN 6 PRECEDING AND CURRENT ROW -- optional frame  
)
```

02 • RANK vs DENSE_RANK vs ROW_NUMBER

Scores sorted high to low, with a tie at 80:

Score	ROW_NUMBER	RANK	DENSE_RANK
90	1	1	1
80	2	2	2
80	3	2	2
70	4	4	3

Function	Ties	Gaps after a tie	Use when
ROW_NUMBER	Breaks them arbitrarily	No	Exactly one row per group (dedupe, Q6)
RANK	Same rank	Yes (1, 2, 2, 4)	Keep all tied rows (Q8, Q13)
DENSE_RANK	Same rank	No (1, 2, 2, 3)	Nth distinct value (Q4, Q20)

03 • LAG, LEAD and frames

- LAG(x, 1) reads the previous row's value; LEAD(x, 1) reads the next one.
- **Default frame** with ORDER BY: from the first row to the current row, which is why SUM() OVER (ORDER BY d) is a running total.
- ROWS counts physical rows; RANGE works on values (dates), so gaps in data behave differently (Q16).

♦ Simple Analogy

PARTITION BY splits students into classrooms. ORDER BY lines each class up by marks. The function then gives every student a rank, a running total or a look at the neighbour, and nobody leaves the room.

Q1 — Second Highest Salary

Find the second highest salary without using LIMIT, OFFSET or TOP.

Q1.1 • Input table: Employees

EmpID	Name	Salary
1	Alice	60000
2	Bob	80000
3	Charlie	75000
4	David	80000
5	Eve	90000

Q1.2 • SQL query

```
SELECT MAX(Salary) AS Second_Highest_Salary
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary) FROM Employees
);
```

Q1.3 • Expected output

Second_Highest_Salary

80000

Q1.4 • How it works

- 1 The inner query finds the highest salary: **90000** (Eve).
- 2 The outer WHERE keeps only salaries below 90000: 60000, 80000, 75000, 80000.
- 3 MAX of those is **80000**. Bob and David share it, but MAX returns one value, so duplicates do not matter.

♦ Simple Analogy

Remove the race winner from the results, then ask who is fastest now. That person is the runner-up.

★ Alternative: Nth highest

DENSE_RANK() OVER(ORDERBY Salary DESC) in a CTE, then WHERE rnk = N. This generalises to the 3rd, 4th or Nth distinct salary, which is the usual follow-up.

Q2 — Running Total of Revenue

Given a table of orders, write a query to find the running total (cumulative sum) of revenue for each day.

Q2.1 • Input table: Orders

OrderID	OrderDate	Revenue
101	2024-01-01	100
102	2024-01-01	200
103	2024-01-02	150
104	2024-01-03	300
105	2024-01-03	100

Q2.2 • SQL query

```
SELECT
  OrderDate,
  SUM(Revenue) AS Daily_Revenue,
  SUM(SUM(Revenue)) OVER (ORDER BY OrderDate) AS Running_Total_Revenue
FROM Orders
GROUP BY OrderDate
ORDER BY OrderDate;
```

Q2.3 • Expected output

OrderDate	Daily_Revenue	Running_Total_Revenue
2024-01-01	300	300
2024-01-02	150	450
2024-01-03	400	850

Q2.4 • How it works

- 1** **GROUP BY OrderDate** collapses the five orders into three daily totals: 300, 150, 400.
- 2** The inner **SUM(Revenue)** is the daily total. The outer **SUM(...) OVER (ORDER BY OrderDate)** adds every daily total up to and including the current day.
- 3** Result: 300, then $300 + 150 = 450$, then $450 + 400 = 850$.

♦ Simple Analogy

Abankpassbook: each line shows the deposit of the day and the balance so far.

★ Tip

Add **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** to be explicit. Without it, the default RANGE frame treats rows with the same date as peers, and is slower on large tables.

Q3 — Earning More Than the Manager

Write an SQL query to identify employees who earn more than their managers.

Q3.1 • Input table: Employees

EmpID	Name	Salary	ManagerID
1	Alice	70000	NULL
2	Bob	80000	1
3	Charlie	60000	1
4	David	90000	2
5	Eve	85000	2

Q3.2 • SQL query

```
SELECT e.EmpID, e.Name, e.Salary, e.ManagerID
FROM Employees e
JOIN Employees m ON e.ManagerID = m.EmpID
WHERE e.Salary > m.Salary;
```

Q3.3 • Expected output

EmpID	Name	Salary	ManagerID
2	Bob	80000	1
4	David	90000	2
5	Eve	85000	2

Q3.4 • How it works

- 1 Read the same table twice under two aliases: **e** is the employee, **m** is that employee's manager.
- 2 Join on **e.ManagerID = m.EmpID** so each employee row sits next to their boss's row.
- 3 Keep rows where **e.Salary > m.Salary**. Alice has no manager (NULL), so the inner join drops her.

♦ Simple Analogy

Put every employee's card next to their boss's card and compare the salary on the two cards.

★ Check by hand

Bob 80000 > Alice 70000 yes. Charlie 60000 < 70000 no. David 90000 > Bob 80000 yes. Eve 85000 > Bob 80000 yes.

Q4 — Top N Customers, No Duplicates

Find the top N customers whom made the highest purchases, ensuring ties on the same amount are not dropped. Take N = 2 distinct amounts.

Q4.1 • Input table: Purchases

CustomerID	CustomerName	PurchaseAmount
1	Alice	5000
2	Bob	7000
3	Charlie	7000
4	David	6500
5	Eve	6000

Q4.2 • SQL query

```
WITH RankedPurchases AS (  
  SELECT CustomerID, CustomerName, PurchaseAmount,  
         DENSE_RANK() OVER (ORDER BY PurchaseAmount DESC) AS rnk  
  FROM Purchases  
)  
SELECT CustomerID, CustomerName, PurchaseAmount  
FROM RankedPurchases  
WHERE rnk <= 2;
```

Q4.3 • Expected output

CustomerID	CustomerName	PurchaseAmount
2	Bob	7000
3	Charlie	7000
4	David	6500

Q4.4 • Which ranking function?

Function	Ranks given	Result for N = 2
DENSE_RANK	1, 1, 2, 3, 4	Bob, Charlie, David (top 2 distinct amounts)
RANK	1, 1, 3, 4, 5	Bob, Charlie only (David gets rank 3)
ROW_NUMBER	1, 2, 3, 4, 5	Bob, Charlie arbitrarily split; one of the tied pair is lost

★ Key idea

DENSE_RANK gives equal amounts the same rank and never skips a number, so “top 2 distinct amounts” returns every customer tied for those amounts.

Q5 — Consecutive Login Streaks

Identify consecutive login streaks for users where they logged in for at least three consecutive days.

Q5.1 • Input table and the key trick

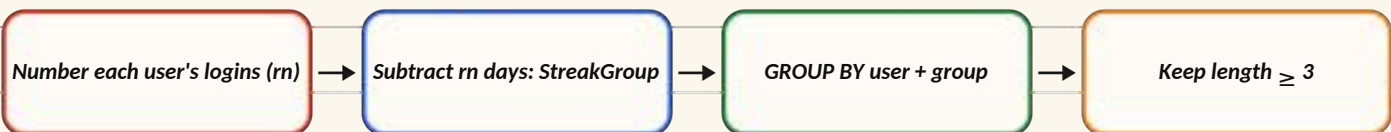
UserID	LoginDate
101	2024-05-01
101	2024-05-02
101	2024-05-03
101	2024-05-05
102	2024-05-10
102	2024-05-11
102	2024-05-12
102	2024-05-13
103	2024-05-15
103	2024-05-17

Worked example, user 101

LoginDate	rn	LoginDate - rn	Group
05-01 05-	1	04-30	A
02 05-03	2	04-30	A
05-05	3	04-30	A
	4	05-01	B

Dates in a run go up by 1 and so does rn, so **date - rn** stays the same for the whole run. A gap in dates makes it jump, which starts a new group.

Q5.2 • The plan in four steps



Q5.3 • Why 103 is excluded and 05-05 does not extend the run

User	Runs found	Longest run	Verdict
101	05-01 to 05-03, then 05-05	3 days	Included ($3 \geq 3$)
102	05-10 to 05-13	4 days	Included
103	05-15, then 05-17 (gap on 05-16)	1 day	Excluded

◆ Simple Analogy

Two people walking up a staircase at the same speed stay the same distance apart. If one stops for a step, the distance changes. A constant gap means an unbroken run.

◆ Where is the query?

Page 11 has the full query, the expected output and the dialect-specific date arithmetic.

Q5 – Streaks – Query and Output

Same problem, continued: the complete solution, its output and the pitfalls to mention.

Q5.1 • SQL query (MySQL style)

```
WITH RankedLogins AS (  
    SELECT UserID, LoginDate,  
           ROW_NUMBER() OVER (PARTITION BY UserID ORDER BY LoginDate) AS rn  
    FROM UserLogins  
)  
,  
Streaks AS (  
    SELECT UserID, LoginDate,  
           DATE_SUB(LoginDate, INTERVAL rn DAY) AS StreakGroup  
    FROM RankedLogins  
)  
,  
GroupedStreaks AS (  
    SELECT UserID,  
           MIN(LoginDate) AS StartDate,  
           MAX(LoginDate) AS EndDate,  
           COUNT(*) AS StreakLength  
    FROM Streaks  
    GROUP BY UserID, StreakGroup  
)  
SELECT *  
FROM GroupedStreaks  
WHERE StreakLength >= 3;
```

Q5.2 • Expected output

UserID	StartDate	EndDate	StreakLength
101	2024-05-01	2024-05-03	3
102	2024-05-10	2024-05-13	4

Q5.3 • Date arithmetic by dialect

Dialect	StreakGroup expression
MySQL	DATE_SUB(LoginDate, INTERVAL rn DAY)
PostgreSQL	LoginDate - rn * INTERVAL '1 day' (for DATE: LoginDate - rn::int)
SQL Server	DATEADD(DAY, -rn, LoginDate)

⚠ Pitfall: two logins on the same day

If a user can login twice a day, rn grows faster than the date and the trick breaks. Select **DISTINCT UserID, LoginDate** (or the date part of a timestamp) in the first CTE.

★ Follow-up you may get

"Longest streak per user?" Replace the final filter with MAX(Streak Length) grouped by UserID.

Q6 — Delete Duplicate Rows

Write an efficient query to detect duplicate records and delete only the extras, keeping one copy. Duplicates are defined by Name+ Email.

Q6.1 • Input table: Customers

CustomerID	Name	Email
1	Alice	alice@example.com
2	Bob	bob@example.com
3	Alice	alice@example.com
4	Charlie	charlie@email.com
5	Alice	alice@example.com

After deletion

CustomerID	Name	Email
1	Alice	alice@example.com
2	Bob	bob@example.com
4	Charlie	charlie@email.com

Q6.2 • Method A: keep MIN(CustomerID) per group

```
DELETE FROM Customers
WHERE CustomerID NOT IN (
    SELECT MIN(CustomerID)
    FROM Customers
    GROUP BY Name, Email
);
```

Q6.3 • Method B: ROW_NUMBER() (PostgreSQL / SQL Server)

```
WITH Duplicates AS (
    SELECT CustomerID,
           ROW_NUMBER() OVER (PARTITION BY Name, Email ORDER BY CustomerID) AS rn
    FROM Customers
)
DELETE FROM Customers
WHERE CustomerID IN (SELECT CustomerID FROM Duplicates WHERE rn > 1);
```

Q6.4 • Detect first, delete second

```
SELECT Name, Email, COUNT(*) AS copies
FROM Customers
GROUP BY Name, Email
HAVING COUNT(*) > 1;      -- Alice, alice@example.com, 3
```

⚠ MySQL error 1093

MySQL refuses to delete from a table that the subquery also reads. Wrap the subquery in a derived table: **NOT IN (SELECT keep_id FROM (SELECT MIN(CustomerID) AS keep_id FROM Customers GROUP BY Name, Email) t).**

★ Good habits

Run the SELECT version first, wrap the DELETE in a transaction, then add a **UNIQUE (Name, Email)** constraint so duplicates cannot return.

Q7 — Optimise a Slow Query (1/2)

You have a table with millions of records. How would you optimise a slow query with multiple joins and aggregations?

Q7.1 • 1 • Add proper indexes

Index the columns used in **JOIN** and **WHERE**. A composite index helps when both appear together.

```
CREATE INDEX idx_customer_id ON Orders(customer_id);
CREATE INDEX idx_order_date ON Orders(order_date);
```

Q7.2 • 2 • Read the EXPLAIN plan

Measure before you change anything. Look for full table scans, row estimates that are far off, and joins that build huge intermediate results.

```
EXPLAIN ANALYZE
SELECT ...
FROM ...
JOIN ...
WHERE ...;
```

Q7.3 • 3 • Avoid SELECT *

Fetch only the columns you need. Less data moves through joins, sorts and the network, and covering indexes become possible.

```
-- Instead of this
SELECT * FROM Orders;
-- Use this
SELECT OrderID, CustomerID, Revenue FROM Orders;
```

Q7.4 • 4 • Aggregate early with a CTE or temp table

```
WITH RevenuePerCustomer AS (
  SELECT CustomerID, SUM(Revenue) AS TotalRev
  FROM Orders
  GROUP BY CustomerID
)
SELECT c.CustomerName, r.TotalRev
FROM RevenuePerCustomer r
JOIN Customers c ON c.CustomerID = r.CustomerID;
```

◆ Good to know

CTE mainly improves readability. In PostgreSQL 12+ a CTE used once is inlined, so if you need the intermediate result to be computed exactly once, use a temp table or MATERIALIZED.

Q7 — Optimise a Slow Query (2/2)

Continued: join strategy, partitioning, pre-computation and the checklist to quote in an interview.

Q7.1 • 5 • Join order and early filtering

Reduce rows before you join. Filter and aggregate the big table first, then join the small result to dimension tables. Modern optimisers reorder joins themselves, so your real lever is writing selective filters and keeping statistics fresh.

Q7.2 • 6 • Partitioning and batching

- Partition large fact tables by date so a one-month query reads one partition, not the whole table.
- For maintenance scripts, delete or insert in batches (for example 10,000 rows at a time) to avoid long locks.

Q7.3 • 7 • Materialised views or summary tables

```
CREATE MATERIALIZED VIEW MonthlyRevenue AS
SELECT CustomerID, DATE_TRUNC('month', OrderDate) AS Month, SUM(Revenue) AS TotalRevenue
FROM Orders
GROUP BY CustomerID, DATE_TRUNC('month', OrderDate);    -- PostgreSQL / Oracle
```

MySQL has no materialised views: build a summary table and refresh it with a scheduled job.

Q7.4 • 8 • Analytic functions instead of repeated subqueries

One window function pass often replaces a correlated subquery that runs once per row.

Q7.5 • Symptom to fix

Symptom in EXPLAIN	Likely fix
Seq Scan / full table scan on a big table	Index on the filter or join column
Function on an indexed column (YEAR(d) = 2024)	Range filter: d >= '2024-01-01' AND d < '2025-01-01'
Huge rows returned from a join	Filter or aggregate before joining
Same heavy aggregate run every day	Materialised view or summary table
Correlated subquery per row	Window function or join to a pre-aggregated CTE

★ Final tip

Combine EXPLAIN evidence, the right indexes, column pruning and early aggregation. Always measure before and after.

Q8 — First Order per Customer

Retrieve the first order for each customer, ensuring that ties(multiple first orders on the same date)are handled correctly.

Q8.1 • Input table: Orders

OrderID	CustomerID	OrderDate	Amount
1	101	2024-05-01	500
2	101	2024-05-01	450
3	101	2024-05-03	600
4	102	2024-05-02	700
5	103	2024-05-01	300
6	103	2024-05-01	350

Q8.2 • SQL query using RANK()

```
WITH RankedOrders AS (  
    SELECT *,  
           RANK() OVER (PARTITION BY CustomerID ORDER BY OrderDate ASC) AS rnk  
    FROM Orders  
)  
SELECT OrderID, CustomerID, OrderDate, Amount  
FROM RankedOrders  
WHERE rnk = 1;
```

Q8.3 • Expected output

OrderID	CustomerID	OrderDate	Amount
1	101	2024-05-01	500
2	101	2024-05-01	450
4	102	2024-05-02	700
5	103	2024-05-01	300
6	103	2024-05-01	350

Q8.4 • Ties: all of them, or just one?

Requirement	Use	Result
Every order on the first date	RANK() or DENSE_RANK()	5 rows above
Exactly one row per customer	ROW_NUMBER() ... ORDER BY OrderDate, OrderID	Orders 1, 4, 5 (the tiebreaker makes it repeatable)

◆ Simple Analogy

Two people join the queue at 9:00 sharp. RANK says both are first. ROW_NUMBER needs a tiebreaker, such as who signed in first.

Q9 — Products Never Purchased

Find products that were never purchased by any customer.

Q9.1 • Input tables

Products

ProductID	ProductName
1	Shoes
2	Bag
3	Watch
4	Hat

OrderItems

OrderID	ProductID	Quantity
101	1	2
102	3	1
103	1	1

Q9.2 • SQL query using LEFT JOIN

```
SELECT p.ProductID, p.ProductName
FROM Products p
LEFT JOIN OrderItems o ON p.ProductID = o.ProductID
WHERE o.ProductID IS NULL;
```

Q9.3 • Expected output

ProductID	ProductName
2	Bag
4	Hat

Q9.4 • Three ways to write an anti-join

```
-- NOT EXISTS (safe with NULLs, usually the best plan)
SELECT p.* FROM Products p
WHERE NOT EXISTS (SELECT 1 FROM OrderItems o WHERE o.ProductID = p.ProductID);

-- NOT IN (only safe if the subquery column can never be NULL)
SELECT * FROM Products WHERE ProductID NOT IN (SELECT ProductID FROM OrderItems);
```

⚠ The NOT IN trap

If OrderItems.ProductID contains even one NULL, NOT IN returns no rows at all, because comparing with NULL is unknown. Prefer NOT EXISTS or LEFT JOIN ... IS NULL.

◆ Simple Analogy

Lay the full catalog next to the sales receipts and circle every item that never appears on a receipt.

Q10 — Active in Every Month

Given a table of customer transactions, find customers who made transactions in every month of the year (2024).

Q10.1 • Input table: Transactions (sample)

TransactionID	CustomerID	TransactionDate
1	101	2024-01-15
2	101	2024-02-10
3	101	2024-03-20
...	...	... (one row for each month up to December)
13	101	2024-12-01
14	102	2024-01-05
15	102	2024-04-17

Q10.2 • SQL query

```
SELECT CustomerID
FROM (
    SELECT CustomerID,
           EXTRACT(MONTH FROM TransactionDate) AS txn_month
    FROM Transactions
    WHERE EXTRACT(YEAR FROM TransactionDate) = 2024
    GROUP BY CustomerID, EXTRACT(MONTH FROM TransactionDate)
) AS monthly_txns
GROUP BY CustomerID
HAVING COUNT(DISTINCT txn_month) = 12;
```

Q10.3 • Expected output

CustomerID
101

Q10.4 • Shorter and index-friendly version

```
SELECT CustomerID
FROM Transactions
WHERE TransactionDate >= '2024-01-01' AND TransactionDate < '2025-01-01' GROUP
BY CustomerID
HAVING COUNT(DISTINCT EXTRACT(MONTH FROM TransactionDate)) = 12;
```

★ Key idea

Count **distinctmonths**, not transactions. A customer with 40 transactions in 11 months still fails. Replacing the YEAR() function with a date range lets an index on TransactionDate be used.

⚠ Edge case

Customer102 has only two months (January, April), so $2 < 12$ and they are excluded.

Q11 – Same Salary in the Same Department

Find employees who have the same salary as another employee in the same department.

Q11.1 • Input table: Employees

EmpID	EmpName	Department	Salary
1	Alice	Sales	50000
2	Bob	Sales	60000
3	Carol	Sales	50000
4	David	HR	45000
5	Eva	HR	45000
6	Frank	HR	47000

Q11.2 • SQL query (self join)

```
SELECT e1.EmpID, e1.EmpName, e1.Department, e1.Salary
FROM Employees e1
JOIN Employees e2
  ON e1.Department = e2.Department
 AND e1.Salary     = e2.Salary
 AND e1.EmpID      <> e2.EmpID;
```

Q11.3 • Expected output

EmpID	EmpName	Department	Salary
1	Alice	Sales	50000
3	Carol	Sales	50000
4	David	HR	45000
5	Eva	HR	45000

Q11.4 • Window-function alternative

```
SELECT EmpID, EmpName, Department, Salary
FROM (
  SELECT *, COUNT(*) OVER (PARTITION BY Department, Salary) AS same_pay
  FROM Employees
) t
WHERE same_pay > 1;
```

⚠ Duplicate rows with the self join

If three people share a salary, each matches two others and appears twice. Add **DISTINCT**, or use the **COUNT() OVER** version, which returns each person once.

Q12 – Department with the Highest Total Salary

Retrieve the department with the highest total salary paid to employees.

Q12.1 • Input table: Employees

EmpID	EmpName	Department	Salary
1	Alice	Sales	50000
2	Bob	Sales	60000
3	Carol	HR	45000
4	David	HR	47000
5	Eva	Tech	90000
6	Frank	Tech	80000

After GROUP BY Department

Department	TotalSalary
Sales	110000
HR	92000
Tech	170000

Q12.2 • SQL query (ORDER BY + LIMIT)

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
ORDER BY TotalSalary DESC
LIMIT 1;
```

Q12.3 • Alternative with RANK() (no LIMIT)

```
WITH DeptSalaries AS (
    SELECT Department, SUM(Salary) AS TotalSalary
    FROM Employees
    GROUP BY Department
),
RankedDepts AS (
    SELECT *, RANK() OVER (ORDER BY TotalSalary DESC) AS rnk
    FROM DeptSalaries
)
SELECT Department, TotalSalary
FROM RankedDepts
WHERE rnk = 1;
```

Q12.4 • Expected output

Department	TotalSalary
Tech	170000

⚠ Ties

LIMIT 1 silently drops a second department with the same total. **RANK() = 1** keeps both. SQL Server uses **SELECT TOP 1** instead of **LIMIT**.

Q13 — Top 3 Orders per Customer

Rank orders based on order value for each customer and return only the top 3 orders per customer.

Q13.1 • Input table: Orders

OrderID	CustomerID	OrderValue
1	101	1000
2	101	1500
3	101	1200
4	101	800
5	102	700
6	102	1000
7	102	1200
8	102	1300

Q13.2 • SQL query

```
WITH RankedOrders AS (  
  SELECT *,  
         RANK() OVER (PARTITION BY CustomerID ORDER BY OrderValue DESC) AS rnk  
  FROM Orders  
)  
SELECT OrderID, CustomerID, OrderValue  
FROM RankedOrders  
WHERE rnk <= 3;
```

Q13.3 • Expected output

OrderID	CustomerID	OrderValue
2	101	1500
3	101	1200
1	101	1000
8	102	1300
7	102	1200
6	102	1000

★ Pattern name: top-N per group

PARTITIONBY the group, ORDERBY the measure DESC, rank in a CTE, filter rank <= N outside. Customer 101's 800 order is rank 4, so it is dropped. Use **ROW_NUMBER** for exactly 3 rows per customer, **RANK** to keep ties, **DENSE_RANK** for the top 3 distinct values.

Q14 — Median Salary Without Built-ins

Find the median salary of employees using SQL, without built-in median functions.

Q14.1 • Input table: Employees

EmpID	EmpName	Salary
1	Alice	50000
2	Bob	60000
3	Carol	55000
4	David	70000
5	Eva	75000

Sorted, with rn

Salary	rn	total_count
50000	1	5
55000	2	5
60000	3	5
70000	4	5
75000	5	5

Q14.2 • SQL query (works for odd and even counts)

```
WITH RankedSalaries AS (  
  SELECT Salary,  
         ROW_NUMBER() OVER (ORDER BY Salary) AS rn,  
         COUNT(*) OVER () AS total_count  
  FROM Employees  
)  
SELECT ROUND(AVG(Salary), 2) AS MedianSalary  
FROM RankedSalaries  
WHERE rn IN ((total_count + 1) / 2, (total_count + 2) / 2);
```

Q14.3 • Expected output

MedianSalary
60000.00

Q14.4 • Which rows are the middle?

Rows	$(n + 1) / 2$	$(n + 2) / 2$	Rows averaged
5 (odd)	3	3	Row 3 only: 60000
4 (even)	2	3	Rows 2 and 3, averaged

⚠ Bug to avoid

A common version filters $rn = (n+1)/2$ OR $rn = n/2$ OR $rn = n/2 + 1$. With integer division and $n = 5$, $n/2$ is 2, so it also picks row 2 and returns 57500 instead of 60000. The formula above is safe.

★ When built-ins exist

Postgre SQL and Oracle have `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Salary)`. MySQL does not, so interviewers ask for this version.

Q15 — Pivot Sales into Months

Pivot a table where each row is a sales transaction into a summary where each column represents a month.

Q15.1 • Input table: Sales

TransactionID	CustomerID	SaleAmount	SaleDate
1	101	1000	2024-01-15
2	101	1500	2024-02-12
3	101	700	2024-03-10
4	102	1200	2024-01-10
5	102	800	2024-03-21

Q15.2 • SQL query: CASE + SUM

```
SELECT
  CustomerID,
  SUM(CASE WHEN MONTH(SaleDate) = 1 THEN SaleAmount ELSE 0 END) AS Jan,
  SUM(CASE WHEN MONTH(SaleDate) = 2 THEN SaleAmount ELSE 0 END) AS Feb,
  SUM(CASE WHEN MONTH(SaleDate) = 3 THEN SaleAmount ELSE 0 END) AS Mar --
  extend up to Dec if needed
FROM Sales
GROUP BY CustomerID;
```

Q15.3 • Expected output

CustomerID	Jan	Feb	Mar
101	1000	1500	700
102	1200	0	800

Q15.4 • Dialect notes

Need	How
Month number	MONTH(d) in MySQL / SQL Server; EXTRACT(MONTH FROM d) in PostgreSQL
NULL instead of 0	Drop ELSE 0, or in PostgreSQL: SUM(x) FILTER (WHERE EXTRACT(MONTH FROM d) = 1)
Built-in pivot	SQL Server has a PIVOT operator; CASE + SUM works everywhere

⚠ Filter the year

If the table spans several years, add **WHERE YEAR(SaleDate) = 2024**, otherwise January 2023 and January 2024 are added into one column.

◆ Simple Analogy

Rotate the table a quarter turn: the rows that were months become column headings.

Q16 — 7-Day Moving Average

Find the moving average of sales for the last 7 days for each product.

Q16.1 • Input table: Sales (product P1)

SaleDate	06-01	06-02	06-03	06-04	06-05	06-06	06-07	06-08
SaleAmount	100	200	150	250	100	300	200	500

Q16.2 • SQL query

```
-- Date-aware frame (MySQL 8, PostgreSQL 11+): handles missing days
SELECT ProductID, SaleDate,
       ROUND(AVG(SaleAmount) OVER (
         PARTITION BY ProductID ORDER BY SaleDate
         RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW), 2) AS MovingAvg7Days
FROM Sales;

-- Row-based frame: assumes exactly one row per product per day
SELECT ProductID, SaleDate,
       ROUND(AVG(SaleAmount) OVER (
         PARTITION BY ProductID ORDER BY SaleDate
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 2) AS MovingAvg7Days
FROM Sales;
```

Q16.3 • Expected output for P1

SaleDate	06-01	06-02	06-03	06-04	06-05	06-06	06-07	06-08
MovingAvg7Days	100.00	150.00	150.00	175.00	160.00	183.33	185.71	242.86

Q16.4 • Check the last value

On 06-08 the window is 06-02 to 06-08: $200 + 150 + 250 + 100 + 300 + 200 + 500 = 1700$, and $1700 / 7 = 242.86$. Early days have fewer than 7 rows, so they average whatever exists (06-04: $700 / 4 = 175$).

⚠ ROWS vs RANGE

ROWS 6 PRECEDING means the previous 6 rows, not 6 days. If a product has no sale on some day, the window silently stretches further back in time. Use the RANGE version, or join to a calendar table that fills in missing days.

Q17 — First and Last Occurrence of an Event

Find the first and last occurrence of each event per user.

Q17.1 • Input table: Events

UserID	EventName	EventTime
1	login	2024-05-01 10:00:00
1	login	2024-05-01 15:00:00
1	logout	2024-05-01 18:00:00
2	login	2024-05-02 09:00:00
2	login	2024-05-02 10:30:00
2	logout	2024-05-02 11:30:00
2	logout	2024-05-02 12:00:00

Q17.2 • SQL query

```
SELECT
  UserID,
  EventName,
  MIN(EventTime) AS FirstOccurrence,
  MAX(EventTime) AS LastOccurrence
FROM Events
GROUP BY UserID, EventName;
```

Q17.3 • Expected output

UserID	EventName	FirstOccurrence	LastOccurrence
1	login	2024-05-01 10:00:00	2024-05-01 15:00:00
1	logout	2024-05-01 18:00:00	2024-05-01 18:00:00
2	login	2024-05-02 09:00:00	2024-05-02 10:30:00
2	logout	2024-05-02 11:30:00	2024-05-02 12:00:00

Q17.4 • When you need the whole row

MIN and MAX return only the time. If you also need other columns from that row (device, page), rank the rows instead:

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY UserID, EventName ORDER BY EventTime ASC) AS first_rn,
    ROW_NUMBER() OVER (PARTITION BY UserID, EventName ORDER BY EventTime DESC) AS last_rn
  FROM Events
) t
WHERE first_rn = 1 OR last_rn = 1;
```

⚠ LAST_VALUE trap

LAST_VALUE() uses the default frame that ends at the current row, so it just returns the current row. Add **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**.

Q18 – Two Consecutive Months, Not a Third

Identify users who placed an order in two consecutive months but not in the third month.

Q18.1 • Input table and what the question means

UserID	OrderDate
101	2023-01-15
101	2023-02-10
101	2023-03-05
102	2023-04-12
102	2023-05-18
103	2023-06-20
103	2023-07-25
103	2023-09-03

Read it as: the user ordered in **month M and M+1**, did **not** order in M+2, and the pair is not part of a longer run. So a three-month streak (user 101) does not count.

★ Expected output

UserID 102 and 103.

Q18.2 • Timeline view (orders by month, 2023)

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	
User 101	●	●	●							3-month streak: excluded
User 102				●	●					pair, nothing around: included
User 103						●	●		●	Jun-Jul pair: included

Q18.3 • Solution logic

- 1 Reduce each order to a **month index**: $\text{year} \times 12 + \text{month}$. Then March 2023 and April 2023 differ by exactly 1, even across a year boundary.
- 2 Keep one row per user and month (an order twice in a month still counts once).
- 3 Find pairs where month M and M + 1 both exist.
- 4 Keep the pair only if M - 1 and M + 2 do not exist.

⚠ Why not the LAG / LEAD version from the source?

It subtracts DATE_PART('month') values, which breaks from December to January ($12 \rightarrow 1 = -11$), and it also accepts the last month of a longer streak. For user 101 the March row has a previous month and no next month, so 101 would wrongly be returned.

Q18 — Months Query and Walkthrough

Same problem, continued: the corrected query, a row-by-row check and the island alternative.

Q18.1 • SQL query

```
WITH m AS (  
    SELECT DISTINCT UserID,  
        EXTRACT(YEAR FROM OrderDate) * 12 + EXTRACT(MONTH FROM OrderDate) AS mi  
    FROM Orders  
)  
SELECT DISTINCT a.UserID  
FROM m a  
JOIN      m b ON b.UserID = a.UserID AND b.mi = a.mi + 1 -- second month of the pair  
LEFT JOIN m p ON p.UserID = a.UserID AND p.mi = a.mi - 1 LEFT JOIN -- month before the pair  
m n ON n.UserID = a.UserID AND n.mi = a.mi + 2 WHERE p.mi IS -- month after the pair  
NULL  
AND n.mi IS NULL;
```

Q18.2 • Expected output

UserID
102
103

Q18.3 • Row-by-row check

User	Pair (M, M+1)	Month before?	Month after?	Verdict
101	Jan, Feb	No	Yes (Mar)	Rejected
101	Feb, Mar	Yes (Jan)	No	Rejected
102	Apr, May	No	No	Accepted
103	Jun, Jul	No	No (Aug missing)	Accepted

Q18.4 • Alternative: reuse the Q5 island trick

Number each user's months with ROW_NUMBER, compute $mi - rn$ as the group key, GROUP BY user and key, and keep groups where COUNT(*) = 2. This is the same gaps-and-islands idea, applied to months instead of days.

◆ MySQL month index

Use YEAR(OrderDate)*12+ MONTH(OrderDate) instead of EXTRACT. The rest of the query is unchanged.

★ Key idea

Turn calendar months into plain integers. Consecutive months become consecutive numbers, and all the date logic turns into simple +1 / -1 joins.

Q19 — Most Frequent Category per User

Find the most frequently purchased product category by each user over the past year.

Q19.1 • Input table: Orders

UserID	OrderDate	Category
101	2024-07-01	Electronics
101	2024-07-15	Fashion
101	2024-08-01	Electronics
102	2024-07-10	Grocery
102	2024-08-20	Grocery
102	2024-09-05	Fashion
103	2024-07-01	Fashion
103	2024-07-10	Fashion
103	2024-08-01	Fashion

After COUNT per user and category

UserID	Category	Count
101	Electronics	2
101	Fashion	1
102	Grocery	2
102	Fashion	1
103	Fashion	3

Q19.2 • SQL query

```
WITH LastYearOrders AS (  
    SELECT * FROM Orders  
    WHERE OrderDate >= CURRENT_DATE - INTERVAL '12 months'  
),  
CategoryCounts AS (  
    SELECT UserID, Category, COUNT(*) AS CategoryCount  
    FROM LastYearOrders  
    GROUP BY UserID, Category  
),  
Ranked AS (  
    SELECT *, RANK() OVER (PARTITION BY UserID ORDER BY CategoryCount DESC) AS rnk  
    FROM CategoryCounts  
)  
SELECT UserID, Category, CategoryCount  
FROM Ranked  
WHERE rnk = 1;
```

Q19.3 • Expected output

UserID	Category	CategoryCount
101	Electronics	2
102	Grocery	2
103	Fashion	3

⚠ Sample dates are in 2024

Run today, CURRENT_DATE minus 12 months is late 2025, so the 2024 sample rows would be filtered out. Replace CURRENT_DATE with a fixed date such as DATE '2025-03-01' to reproduce the output. MySQL: DATE_SUB(CURRENT_DATE, INTERVAL 12 MONTH).

Q20 — Rank Products by Sales, Reset Yearly

Write a query to generate a sequential ranking of products based on total sales, but reset the ranking for each year.

Q20.1 • Input table and yearly totals

ProductID	SaleAmount	SaleDate	YearlySales (step 2)		
P1	500	2022-01-15	Year	Product	TotalSales
P2	700	2022-03-22	2022	P1	800
P1	300	2022-05-20	2022	P2	700
P3	800	2022-07-01	2022	P3	800
P1	1000	2023-02-10	2023	P1	1000
P2	600	2023-03-15	2023	P2	600
P3	900	2023-04-12	2023	P3	900

Q20.2 • SQL query

```
WITH YearlySales AS (  
    SELECT EXTRACT(YEAR FROM SaleDate) AS SalesYear, ProductID, SUM(SaleAmount) AS TotalSales  
    FROM Sales  
    GROUP BY EXTRACT(YEAR FROM SaleDate), ProductID  
,  
    RankedProducts AS (  
        SELECT *,  
            RANK() OVER (PARTITION BY SalesYear ORDER BY TotalSales DESC) AS SalesRank,  
            DENSE_RANK() OVER (PARTITION BY SalesYear ORDER BY TotalSales DESC) AS DenseRank  
        FROM YearlySales  
    )  
SELECT * FROM RankedProducts  
ORDER BY SalesYear, SalesRank, ProductID;
```

Q20.3 • Expected output

SalesYear	ProductID	TotalSales	SalesRank	DenseRank
2022	P1	800	1	1
2022	P3	800	1	1
2022	P2	700	3	2
2023	P1	1000	1	1
2023	P3	900	2	2
2023	P2	600	3	3

⚠ Correction to the source output

The source table listed 2022 as P3 800 / P2 700 / P1 800 with mixed ranks. P1 and P3 both total 800 (500 + 300 and 800), so both are rank 1. With RANK the next product (P2) is rank 3; with DENSE_RANK it is 2.

Pattern Map — Which Tool Solves What

Twenty questions, nine patterns. In the room, name the pattern first; the query follows almost automatically.

01 • Nine patterns

Pattern	Core idea	Questions	Key syntax
Subquery + MAX	Remove the top, then take the top again	Q1	WHERE x < (SELECT MAX(x))
Running and moving windows	Aggregate over a growing or sliding frame	Q2, Q16	SUM/AVG OVER (ORDER BY ... ROWS/RANGE)
Self join	Compare a row with another row of the same table	Q3, Q11	FROM T a JOIN T b ON ...
Ranking	Rank inside a group, then filter the rank	Q4, Q8, Q12, Q13, Q19, Q20	RANK / DENSE_RANK / ROW_NUMBER
Gaps and islands	Value minus row number is constant in a run	Q5, Q18	date - rn, month index
Dedupe and anti-join	Keep one copy; find what has no match	Q6, Q9	MIN(id), NOT EXISTS, LEFT JOIN IS NULL
Conditional aggregation	Count or pivot with CASE, filter groups	Q10, Q15, Q17	HAVING, SUM(CASE WHEN ...)
Statistics by hand	Use row positions to find the middle	Q14	ROW_NUMBER + COUNT() OVER ()
Performance	Reduce the data the engine must touch	Q7	Index, EXPLAIN, partition, MV

♦ Simple Analogy

A mechanic does not invent a new tool for every car. Nine tools in the box open nearly every engine.

02 • Questions by pattern at a glance

Ranking: 6 questions

Windows: 2

Self join: 2

Islands: 2

Conditional aggregation: 3

★ Remember

If the question says **top**, **first**, **per group** or **ties**, think ranking. If it says **consecutive**, think islands. If it says **never** or **missing**, think anti-join.

Cheat-sheet — Dialect Differences

The logic is the same everywhere; only the function names change. Window functions need MySQL 8.0 or newer.

01 • Same task, three spellings

Task	MySQL	PostgreSQL	SQL Server
Limit rows	LIMIT n	LIMIT n	TOP n, or OFFSET ... FETCH
Subtract days	DATE_SUB(d, INTERVAL n DAY)	d - n * INTERVAL '1 day'	DATEADD(DAY, -n, d)
Get the month	MONTH(d)	EXTRACT(MONTH FROM d)	MONTH(d)
Start of month	DATE_FORMAT(d, '%Y-%m-01')	DATE_TRUNC('month', d)	DATETRUNC(month, d) in 2022+
Today	CURDATE()	CURRENT_DATE	CAST(GETDATE() AS date)
Median	Window trick (Q14)	PERCENTILE_CONT(0.5)	PERCENTILE_CONT(0.5) OVER
Conditional sum	SUM(CASE ...)	SUM(CASE ...) or FILTER	SUM(CASE ...) or PIVOT
Delete using a CTE	Not supported; use a derived table	Allowed (join / IN)	DELETE straight from the CTE
Materialised view	None; summary table + job	CREATE MATERIALIZED VIEW	Indexed view

⚠ Reserved words

Avoid **Rank**, **Month** or **Year** as column aliases. Several engines treat them as keywords, which is why these pages use **rnk**.

★ In an interview

Say which dialect you are assuming, give the query, and mention the one function you would change for another engine.

Common Mistakes and Pitfalls

Most wrong answers are not wrong ideas; they are right ideas that fail on ties, NULLs or dates. Run this checklist before you submit.

01 • Eight traps

Trap	What goes wrong	Fix	Seen in
ROW_NUMBER with ties	One of two equal rows is dropped	RANK or DENSE_RANK	Q4, Q8, Q13
NOT IN with NULL	Returns zero rows	NOT EXISTS or LEFT JOIN	Q9
Window in WHERE	Syntax error	Rank in a CTE, filter outside	Q4, Q13, Q19
ROWS on gappy dates	Window spans more than 7 days	RANGE with INTERVAL, or calendar table	Q16
Integer division	Wrong middle row for odd counts	$(n+1)/2$ and $(n+2)/2$	Q14
Duplicate dates per day	Islands trick breaks	DISTINCT before ROW_NUMBER	Q5
Delete without a preview	Wrong rows removed	SELECT first, use a transaction	Q6
Function on indexed column	Index ignored	Range filter on the raw column	Q7, Q10

⚠ Sample data lies

Small tables hide problems. Always ask: what if two rows tie? what if the value is NULL? what if a day or month is missing? what if the table has 100 million rows?

02 • Before you press Run

Ties?

NULLs?

Duplicates?

Missing dates?

Index used?

Dialect?

Interview Playbook

A correct query is half the answer. The other half is showing how you think: clarify, build simply, test, then improve.

01 • Answer in six moves

- 1 Clarify:** ties, NULLs, duplicates, which SQL dialect, expected table size.
- 2 Name the pattern:** "This is top-N per group" or "This is gaps and islands."
- 3 Write the simple version first,** even if it is not the fastest.
- 4 Trace the sample** by hand and say the output out loud.
- 5 Handle edge cases:** the tie, the missing month, the NULL.
- 6 Discuss performance:** which index, what EXPLAIN would show.

02 • Follow-ups to expect

After	They may ask
Q1	Now the Nth highest salary. (DENSE_RANK = N)
Q2	Reset the running total each month. (PARTITION BY month)
Q5	Longest streak per user; a streak that allows a one-day gap.
Q8 / Q13	What changes if you want exactly one row per customer? (ROW_NUMBER with a tiebreaker)
Q9	Why is NOT EXISTS safer than NOT IN?
Q14	Median per department. (Add PARTITION BY to both window functions)
Q16	What if some days have no sales? (RANGE frame or calendar table)
Q19	What if two categories tie for first?

★ Say your trade-offs

"I used a CTE for readability; on a 100 million row table I would check the plan and index CustomerID and OrderDate."

Final Summary

01 • The twenty answers in one table

Q	One-line solution
Q1	MAX of salaries below the overall MAX
Q2	SUM(SUM(Revenue)) OVER (ORDER BY date) after GROUP BY date
Q3	Self join on ManagerID = EmpID, filter employee salary > manager salary Rank in a CTE (DENSE_RANK / RANK), filter the
Q4, Q8, Q13 Q5	rank outside
Q6	date - ROW_NUMBER() as group key, count each group
Q7	Keep MIN(id) per Name + Email, or delete ROW_NUMBER() > 1
Q9	Index, EXPLAIN, select fewer columns, aggregate early, pre-compute LEFT JOIN ... IS NULL (or NOT EXISTS)
Q10	HAVING COUNT(DISTINCT month) = 12
Q11	Self join on department and salary with different ids
Q12	SUM per department, ORDER BY DESC LIMIT 1 (or RANK)
Q14	rn IN ((n+1)/2, (n+2)/2), then AVG
Q15	SUM(CASE WHEN MONTH(d) = k THEN x ELSE 0 END)
Q16	AVG OVER a 7-day RANGE (or 6 PRECEDING) frame
Q17	MIN and MAX of the timestamp, grouped
Q18	Month index, pair joined with no month before and none after
Q19	COUNT per user and category, RANK = 1 over the last 12 months
Q20	RANK PARTITION BY year ORDER BY total DESC

★ One-Line Understanding

SQL interviews reward **patterns**: rank inside groups, join a table to itself, subtract a row number to find streaks, and always check ties, NULLs and missing dates.

Corrections Log and Thank You

01 • What changed from the source questions

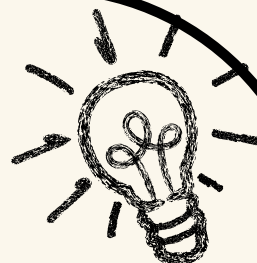
Q	Page	Issue in the source	What this guide does
Q6	12	DELETE ... NOT IN (subquery on same table) fails in MySQL (error 1093) OR-	Adds the derived-table fix
Q14	21	based row filter picks the wrong row for odd counts (57500, not 60000)	Uses (n+1)/2 and (n+2)/2
Q16	23	Printed value for 06-08 was 228.57	Corrected to 242.86 (1700 / 7)
Q18	25-26	LAG/LEAD query returns user 101 and fails across year ends CURRENT_DATE	Month index with no month before or after
Q19	27	filter would exclude the 2024 sample data	Notes a fixed reference date
Q20	28	Printed 2022 rows inconsistent with the totals	Correct output, RANK and DENSE_RANK

Thank you

Create the sample tables, run each query, then change one detail:
add a tie, add a NULL, remove a month. Watch what breaks.

Built from: Myntra Data Analyst Interview, 20 SQL Questions

“



Created by @Ganesh R

I hope you have found this **Document** actionable to
become a **DATA ENGINEER/ANALYST**

Join **with us** to get more updates
stuff Similar to this you finished reading



NO_DATA_TRAIN(AZURE, AWS)



NO_DATA_TRAIN(AZURE, AWS)

”

Thank you



If You Found This Helpful?



Like



Comment



Repost



Share



Save



Download

**Follow for more amazing content
like this !**